

WIRELESS AND MOBILE COMMUNICATION LAB MANUAL

Experiment 1: Study of electromagnetic spectrum allocation and wireless bands used in communication

1. Aim

To study the electromagnetic (EM) spectrum allocation, classify various wireless frequency bands, and analyze their propagation characteristics, applications, and regulatory management models in communication systems.

2. Software Requirement

- **Simulation/Analytical Tool:** MATLAB / GNU Radio / Cisco Packet Tracer (or any standard RF spectrum visualization software tool, if required for verification).
- **System Requirements:** Operating System (Windows/Linux) with standard web browser access for spectrum allocation lookup tables (FCC / ITU databases).

3. Theory

3.1 The Electromagnetic Spectrum

The Electromagnetic Spectrum spans the entire range of electromagnetic radiation arranged according to frequency (f) or wavelength (λ). The relationship between frequency, wavelength, and the speed of light ($c \approx 3 \times 10^8 \text{ m/s}$) is given by:

$$c = f \cdot \lambda$$

3.2 ITU Frequency Band Classification

The International Telecommunication Union (ITU) categorizes the radio frequency (RF) spectrum into distinct functional bands:

Band Name	Frequency Range	Propagation Mode	Primary Applications
Very Low Frequency (VLF)	3 kHz - 30 kHz	Ground/Guided Wave	Submarine & Military Comms
Low Frequency (LF)	30 kHz - 300 kHz	Ground Wave	Long-range radio navigation
Medium Frequency (MF)	300 kHz - 3 MHz	Ground/Skywave	AM Broadcasting
High Frequency (HF)	3 MHz - 30 MHz	Skywave (Ionospheric reflection)	Shortwave, Aviation, Amateur Radio
Very High Frequency (VHF)	30 MHz - 300 MHz	Line-of-Sight (LoS)	FM Radio, Air Traffic Control, TV
Ultra High Frequency (UHF)	300 MHz - 3 GHz	Line-of-Sight	2G/3G/4G Cellular, Wi-Fi 2.4 GHz, GPS
Super High Frequency (SHF)	3 GHz - 30 GHz	Line-of-Sight / Direct	Wi-Fi 5/6, Satellite, Radar
Extremely High Frequency (EHF)	30 GHz - 300 GHz	Short LoS/mmWave	5G mmWave, Radio Astronomy

3.3 Spectrum Allocation Models

Spectrum is a finite natural resource requiring global and national regulatory governance to prevent signal interference:

1. **Licensed Spectrum:** Exclusive frequency blocks assigned to specific operators via government auctions (e.g., Cellular operators for 4G/5G).
2. **Unlicensed Spectrum (ISM Bands):** Industrial, Scientific, and Medical bands open for public use under strict power limits (e.g., 2.4 GHz, 5 GHz, 6 GHz for Wi-Fi and Bluetooth).
3. **Shared Spectrum:** Dynamic access models where secondary users use frequencies when primary users are inactive (e.g., CBRS).

4. Algorithm / Experimental Procedure

Procedure Flowchart Logic:

[Start]

- > Step 1: Identify EM Spectrum Regions (VLF to EHF)
- > Step 2: Relate Frequency (f), Wavelength (λ), and Speed of Light (c)
Formula: $\lambda = c / f$
- > Step 3: Categorize Communication Bands (Cellular, Wi-Fi, Satellite)
- > Step 4: Map Allocation Models (Licensed vs. Unlicensed vs. Shared)
- > Step 5: Analyze Trade-offs (Coverage vs. Capacity)

[End]

1. **Parameter Calculation:** Compute wavelength (λ) for key operating frequencies using $\lambda = c / f$.

◦ Example (Wi-Fi 2.4 GHz): $\lambda = (3 \times 10^8) / (2.4 \times 10^9) = 0.125 \text{ m} = 12.5 \text{ cm}$

◦ Example (5G mmWave 28 GHz): $\lambda = (3 \times 10^8) / (28 \times 10^9) = 0.0107 \text{ m} = 10.7 \text{ mm}$

2. **Trade-off Analysis:**

- Low Frequencies (<1 GHz): Larger wavelength → Better building penetration & long coverage → Lower bandwidth.
- High Frequencies (>24 GHz): Millimeter wavelengths → Poor penetration & short range → Massive data capacity.

5. Conclusion

In this study, the fundamental structure of the electromagnetic spectrum and its allocation across various wireless communication bands were analyzed. It is observed that lower frequency bands offer higher propagation coverage at the expense of lower bandwidth, whereas higher frequency bands (e.g., mmWave) deliver significantly higher data rates over shorter distances. Effective spectrum licensing and governance by bodies like ITU and FCC are essential for preventing interference and maximizing spectral efficiency.

Sources

1. Electromagnetic Spectrum: Radio Waves Explained with Uses
2. Electromagnetic spectrum | Definition, Diagram, & Uses - Britannica

Experiment 2: Simulation of basic communication system (Tx-Channel-Rx) using Python

1. Aim

To simulate a basic digital communication system (Transmitter-Channel-Receiver) using Python, implement Binary Phase Shift Keying (BPSK) modulation over an Additive White Gaussian Noise (AWGN) channel, and evaluate the recovered binary data and Bit Error Rate (BER).

2. Software Requirement

- **Programming Language:** Python 3.x
- **Required Libraries:**
 - NumPy: For random binary sequence generation, vector operations, and noise generation.
 - Matplotlib: For visual analysis of signals at different stages of the system.
- **Development Environment:** Jupyter Notebook / VS Code / PyCharm/Google Colab

3. Theory

3.1 Digital Communication Block Diagram

A fundamental digital communication link is composed of three core stages:

1. **Transmitter (Tx):** Generates digital binary information bits $b_i \in \{0,1\}$ and converts them into physical transmission symbols via modulation.
2. **Channel (AWGN):** The physical propagation medium that introduces random noise. In an Additive White Gaussian Noise channel, thermal noise with a normal distribution is added to the transmitted signal.
3. **Receiver (Rx):** Processes the incoming noisy signal r_i and applies a decision rule to estimate the original transmitted bits $\hat{b}_i$.

3.2 BPSK Modulation & Demodulation Equations

- **Transmitter Mapping:** In BPSK, binary bits are mapped to antipodal symbols $s \in \{-1,+1\}$:
$$s = 2b - 1$$

If bit $b=1 \Rightarrow s=+1$
If bit $b=0 \Rightarrow s=-1$

- **AWGN Channel Model:** The noise standard deviation σ is computed from the target Signal-to-Noise Ratio (SNR):

$$SNR_{linear} = 10^{(SNR_{dB}/10)}$$

$$\sigma = \sqrt{1 / (2 \cdot SNR_{linear})}$$

The received signal vector r is modeled as:

$$r = s + n, \text{ where } n \sim N(0, \sigma^2)$$

- **Receiver Decision Rule:** Demodulation uses zero-threshold detection:

$$\hat{b} = \{ 1 \text{ if } r \geq 0, 0 \text{ if } r < 0 \}$$

4. Algorithm / Experimental Procedure

Procedure Flowchart:

```
[Start]
-> Step 1: Initialize system parameters (num_bits, snr_db, random seed)
-> Step 2: Generate random binary array of size 'num_bits'
-> Step 3: Map binary bits to BPSK symbols: bpsk_symbols = 2 * binary_data - 1
-> Step 4: Convert SNR dB to linear scale and calculate noise standard deviation (sigma)
-> Step 5: Generate Gaussian noise vector and add to symbols: received_signal = bpsk_symbols +
-> Step 6: Apply decision logic at Rx: received_bits = (received_signal >= 0)
-> Step 7: Compute Bit Errors and BER = (bit_errors / num_bits)
-> Step 8: Plot Tx Data, Noisy Channel Signal, and Rx Data using Matplotlib
[End]
```

Python Source Code:

```

import numpy as np
import matplotlib.pyplot as plt

# Step 1: Initialize Parameters
np.random.seed(42)
num_bits = 10
snr_db = 5 # Signal-to-Noise Ratio in dB

# Step 2: Transmitter (Tx)
binary_data = np.random.randint(0, 2, num_bits)
bpsk_symbols = 2 * binary_data - 1 # 0->-1, 1->+1

# Step 3: AWGN Channel
snr_linear = 10**(snr_db/10.0)
noise_std = np.sqrt(1.0 / (2 * snr_linear))
noise = np.random.normal(0, noise_std, num_bits)
received_signal = bpsk_symbols + noise

# Step 4: Receiver (Rx)
received_bits = (received_signal >= 0).astype(int)

# Step 5: Calculate Bit Error Rate (BER)
bit_errors = np.sum(binary_data != received_bits)
ber = bit_errors / num_bits

print(f"Transmitted Bits: {binary_data}")
print(f"BPSK Symbols      : {bpsk_symbols}")
print(f"Received Signal : {np.round(received_signal, 2)}")
print(f"Demodulated Bits: {received_bits}")
print(f"Bit Errors       : {bit_errors}/{num_bits} (BER = {ber})")

# Step 6: Plotting
plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.stem(binary_data, basefmt=" ")
plt.title("Transmitted Binary Data (Tx)")
plt.ylim([-0.2, 1.2])

plt.subplot(3, 1, 2)
plt.plot(received_signal, 'r-o', label='Noisy Received Signal')
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.title(f"Received Signal with AWGN Noise (SNR = {snr_db} dB)")
plt.legend()

plt.subplot(3, 1, 3)
plt.stem(received_bits, basefmt=" ")
plt.title("Demodulated Binary Data (Rx)")
plt.ylim([-0.2, 1.2])
plt.tight_layout()
plt.show()

```

5. Conclusion

A basic digital communication chain (Tx-Channel-Rx) was implemented in Python. The program successfully mapped binary data into BPSK symbols, introduced AWGN channel distortion according to the designated SNR level, and accurately recovered the transmitted bit sequence at the receiver using zero-threshold detection. The visual plots confirm the impact of additive noise on amplitude variation and demonstrate the decision process involved in digital demodulation.

Experiment 3: Demonstration of frequency reuse concept in cellular systems

1. Aim

To study and understand the cellular frequency reuse concept using the IIT Kharagpur Virtual Lab simulator by:

1. Identifying co-channel cells for a designated center cell for given shift parameters (i,j).
2. Identifying and mapping cell clusters (N) within a given geographical cellular network.

2. Software Requirement

- **Platform:** Web Browser (Google Chrome / Mozilla Firefox) with JavaScript enabled.
- **Virtual Lab Simulator:** IIT Kharagpur Wireless Communication Virtual Lab (Fading Channel & Mobile Communications)
- **URL:** <http://vlabs.iitkgp.ac.in/fcmc/exp6/index.html>

3. Theory

3.1 Cellular Frequency Reuse

A fundamental constraint in wireless communications is the limited availability of radio spectrum. To support a large subscriber base across a wide area, the frequency reuse technique is employed.

Base stations are arranged in hexagonal cell layouts. Adjacent cells are assigned distinct channel sets to avoid adjacent-channel interference. However, cells separated by a sufficient geographical distance can reuse the exact same set of channels without causing unacceptable Co-Channel Interference (CCI).

3.2 Hexagonal Geometry & Cluster Size (N)

Hexagonal geometry is chosen because it seamlessly covers a 2D plane without gaps or overlapping areas, providing an optimal approximation of circular radio coverage.

The total number of cells in a cluster (N) is given by the shift parameters i and j ($i, j \geq 0$):

$$N = i^2 + i \cdot j + j^2$$

Common valid cluster sizes include $N = 1, 3, 4, 7, 12, 13, 19$

3.3 Locating Co-Channel Cells

To locate the nearest co-channel neighbors from a reference center cell:

1. Move i cells along any chain of hexagons.
2. Turn 60° counter-clockwise and move j cells.

3.4 Key Equations

- **Total Channel Capacity (C):**

$$C = M \cdot K \cdot N = M \cdot S$$

where S is the total duplex channels, k is channels per cell, N is cluster size, and M is the number of times the cluster is replicated in the network.

- **Co-Channel Reuse Ratio (Q):**

$$Q = D / R = \sqrt{3N}$$

where D is the distance between co-channel cell centers and R is the cell radius.

4. Algorithm / Procedure

Procedure Flowchart:

```
[Start Virtual Lab Simulator]
-> Select Mode: Experiment 6A (Co-Channel Cell) OR Experiment 6B (Cell Cluster)
-> Step 1: Input Parameters (Cell Radius R, Shift Parameters i and j)
-> Step 2: Calculate Cluster Size  $N = i^2 + i*j + j^2$ 
-> Step 3: Click 'Show Cells' to render hexagonal layout
-> Step 4: Identify Co-Channel / Cluster Cells manually (Apply 'Move i cells, turn  $60^\circ$  counter
-> Step 5: Click 'CHECK' to verify selections (Green = Correct, Sky Blue = Corrected)
-> Step 6: Click 'REPORT' to generate and save PDF report
```

Step-by-Step Execution on the Virtual Lab Portal:

Part A: Experiment 6A (Finding Co-Channel Cells)

1. Open the virtual lab simulator link and select Experiment 6A (Co-channel cell). Click START and enter your name.
2. Set the desired Cell Radius (R), along with non-negative integer shift parameters i and j (e.g., $i=2, j=1 \Rightarrow N=7$).
3. Click Show Cells. The center reference cell will be highlighted in Pink.
4. Using the geometric rule (i cells straight, 60° counter-clockwise turn, j cells straight), click on the co-channel cells to mark them in Orange.

5. Click CHECK. Correctly selected co-channel cells will turn Green. Missed or incorrect cells will be displayed in Sky Blue.
6. Click REPORT to save your verified observations.

Part B: Experiment 6B (Cell Cluster Identification)

1. Select Experiment 6B (Cell Cluster) and click START.
2. Input the Cell Radius (R) and select a Cell Cluster (N).
3. Click Show Cells. The boundary cells of the cluster will be shown in Pink.
4. Select all intermediate cells belonging to that cluster so they turn Orange.
5. Click CHECK to confirm your cluster boundary selection (highlighted in Green upon success).
6. Click REPORT to generate the final execution summary.

5. Conclusion

The concepts of frequency reuse, cluster formation, and co-channel cell mapping were verified using the IIT Kharagpur Virtual Lab platform. The experiment confirms that:

1. The co-channel distance $D = R\sqrt{3N}$ increases with a larger cluster size N, which reduces co-channel interference at the cost of lower capacity per individual cell.
2. The shift parameter algorithm (i,j) provides a systematic method to map frequency assignments across hexagonal cellular networks without channel overlap.

Experiment 4: Wi-Fi network scanning using mobile apps (SSID, RSSI, channel mapping).

1. Aim

To perform Wi-Fi network scanning using a mobile application to analyze surrounding wireless networks, evaluate signal strength indicators (RSSI), identify co-channel/adjacent-channel interference, and map optimum non-overlapping frequency channels.

2. Software / Hardware Requirement

- **Hardware:** Android / iOS Smartphone with active Wi-Fi and Location services enabled.
- **Software Application:**
 - Android: NetSpot, Wi-Fi Analyzer (by open-source / Farproc), or Fing.
 - iOS: AirPort Utility (with Wi-Fi Scanner enabled in system settings) or Fing.

3. Theory

3.1 Network Key Parameters

1. **SSID (Service Set Identifier):** The human-readable name assigned to a wireless local area network (WLAN).
2. **BSSID (Basic Service Set Identifier):** The unique hardware MAC address of the Access Point (AP) radio transmitter.
3. **RSSI (Received Signal Strength Indicator):** A power measurement of the received RF signal, expressed in negative decibels relative to one milliwatt (dBm).
 - -30 dBm to -50 dBm: Excellent signal strength.
 - -60 dBm to -67 dBm: Good signal strength (adequate for VoIP and streaming).
 - -70 dBm to -80 dBm: Weak signal (prone to packet loss and high latency).
 - -90 dBm: Noise floor / unusable connection.

3.2 Wi-Fi Frequency Bands and Channel Mapping

- **2.4 GHz Band:** Spans 2.400 GHz - 2.4835 GHz. It consists of 11-13 channels, each 20 MHz wide spaced 5 MHz apart. To prevent adjacent-channel interference, only three non-overlapping channels are used: 1, 6, and 11.

- **5 GHz Band:** Offers broader bandwidth and significantly lower interference with non-overlapping 20 MHz, 40 MHz, 80 MHz, or 160 MHz channels (e.g., Channels 36, 40, 44, 48 in UNII-1).

4. Algorithm / Experimental Procedure

Procedure Flowchart:

```
[Start]
-> Step 1: Install Wi-Fi Analyzer application on smartphone
-> Step 2: Enable Location Services & Wi-Fi permissions
-> Step 3: Initiate real-time Access Point scan
-> Step 4: Record SSID, BSSID, RSSI (dBm), Channel Number, and Frequency Band
-> Step 5: Switch to 'Channel Graph' view to examine overlapping networks
-> Step 6: Identify least congested channel for 2.4 GHz and 5 GHz bands
[End]
```

Step-by-Step Execution:

1. **Setup:** Install Wi-Fi Analyzer on the smartphone. Grant required Location permissions (necessary on mobile OSs to access Wi-Fi hardware scan results).
2. **Scan Execution:** Open the app and allow it to scan for nearby Access Points.
3. **Data Collection:** Navigate to the List View and record the observed network details into a data table:

SSID	BSSID (MAC Address)	RSSI (dBm)	Operating Band	Channel No.	Security Type
Campus_Guest	AA:BB:CC:11:22:33	-52 dBm	2.4 GHz	Channel 1	WPA2-PSK
Lab_5G	AA:BB:CC:44:55:66	-64 dBm	5.0 GHz	Channel 36	WPA3
Home_Wi-Fi	11:22:33:44:55:66	-78 dBm	2.4 GHz	Channel 4	WPA2

1. **Spectrum Mapping:** Switch to the Channel Graph / Spectrum View:
 - Observe how networks operating on adjacent channels (e.g., Channel 3 or 4 in 2.4 GHz) overlap with Channel 1 and 6, causing Adjacent-Channel Interference (ACI).
 - Identify co-located Access Points operating on identical channels causing Co-Channel Interference (CCI).
2. **Channel Selection:** Analyze the channel rating graph to determine the optimal channel with the highest star rating / lowest interference.

5. Conclusion

A Wi-Fi network scan was successfully performed using a mobile analyzer app. The experiment demonstrated that:

1. RSSI values directly represent physical distance and signal attenuation through surrounding obstacles.
2. The 2.4 GHz band exhibits significant channel overlap due to limited non-overlapping spectrum, whereas the 5 GHz band provides higher capacity with minimal co-channel congestion.
3. Choosing non-overlapping channels (1, 6, or 11 in 2.4 GHz) minimizes Co-Channel and Adjacent-Channel Interference, maximizing throughput.

Experiment 5: Measurement of signal strength vs distance for Wi-Fi router

1. Aim

To capture live RSSI signal strength data from a connected Wi-Fi router in real time at user-specified distance intervals, automatically compare live measurements against theoretical Friis Free Space Path Loss (FSPL) curves, and plot the empirical attenuation curve in Python.

2. Software Requirement

1. **Programming Environment:** Python 3.x

2. **Required Libraries:**

- pywifi: To read live RSSI (dBm) signal values from the operating system's wireless card.
- NumPy: For numerical processing and Friis FSPL curve computation.
- Matplotlib: For dynamic graph plotting.

Installation Command:

```
pip install pywifi numpy matplotlib
```

3. Hardware & Experimental Setup

- Identify the target router's SSID (e.g., Home_WiFi).
- Mark measuring positions along a straight path at increments: 1m, 2m, 4m, 6m, 8m, and 10m.
- At each distance marker, run the Python script to log live RSSI data points directly into memory.

4. Mathematical Model for Live Comparison

To evaluate empirical RSSI against theoretical models, Friis' Free Space Path Loss (FSPL) is calculated at frequency $f = 2412$ MHz (2.4 GHz):

$$FSPL(dB) = 20\log_{10}(d) + 20\log_{10}(f) - 27.55$$

$$Theoretical\ RSSI(d) = P_{tx} - FSPL(d)$$

Where P_{tx} is the assumed transmit power ($\approx +20$ dBm for standard residential routers).

5. Interactive Real-Time Python Code

This script pauses at each distance step, prompts you to press ENTER when you move your receiver device to the designated mark, takes 5 sample live scans to average out shadow fading, and plots the live experimental curve against the theoretical Friis curve.

```

import time
import numpy as np
import matplotlib.pyplot as plt
import pywifi

# 1. Initialize Wi-Fi Adapter
def initialize_wifi():
    wifi = pywifi.PyWiFi()
    if len(wifi.interfaces()) == 0:
        raise RuntimeError("No Wi-Fi card detected! Ensure Wi-Fi is enabled on your laptop/PC.")
    iface = wifi.interfaces()[0]
    return iface

# 2. Live Scan Helper to fetch RSSI for a target SSID
def get_live_rssi(iface, target_ssid, num_samples=5):
    rssi_samples = []
    print(f"Gathering {num_samples} live RSSI samples for '{target_ssid}'...")
    for _ in range(num_samples):
        iface.scan()
        time.sleep(1.5) # Allow hardware buffer to refresh
        results = iface.scan_results()
        found = False
        for ap in results:
            if ap.ssid == target_ssid:
                rssi_samples.append(ap.signal) # Signal strength in dBm
                found = True
                break
        if not found:
            print(" [Warning] Target SSID not detected in this scan frame.")
    if not rssi_samples:
        return None
    return np.mean(rssi_samples) # Return averaged live RSSI

# 3. Main Data Collection Loop
def main():
    target_ssid = input("Enter your Wi-Fi Router SSID (Network Name): ").strip()
    if not target_ssid:
        print("SSID cannot be empty.")
        return
    try:
        iface = initialize_wifi()
    except Exception as e:
        print(f"Error: {e}")
        return

    # User defined distances in meters
    distances = [1, 2, 4, 6, 8, 10]
    live_rssi_data = []
    valid_distances = []

    print("
--- Starting Live Signal vs Distance Experiment ---")
    for d in distances:

```

```

        input(f"
>> Move your device to {d} meter(s) from the router, then press ENTER to measure...")
        avg_rssi = get_live_rssi(iface, target_ssid)
        if avg_rssi is not None:
            print(f"Measured Live RSSI at {d}m: {avg_rssi:.2f} dBm")
            live_rssi_data.append(avg_rssi)
            valid_distances.append(d)
        else:
            print(f"Failed to detect '{target_ssid}' at {d}m. Skipping point.")

    if len(live_rssi_data) == 0:
        print("
No live data recorded. Ensure target SSID is correct and nearby.")
        return

    # Convert to numpy arrays
    valid_distances = np.array(valid_distances)
    live_rssi_data = np.array(live_rssi_data)

    # 4. Compute Theoretical Friis Model (2.4 GHz, f = 2412 MHz, Tx Power = 20 dBm)
    freq_mhz = 2412
    tx_power_dbm = 20
    d_model = np.linspace(1, 10, 100)
    fspl = 20 * np.log10(d_model) + 20 * np.log10(freq_mhz) - 27.55
    theoretical_rssi = tx_power_dbm - fspl

    # 5. Plotting Live Empirical vs Theoretical Data
    plt.figure(figsize=(10, 6))
    plt.plot(valid_distances, live_rssi_data, 'ro-', label='Live Measured RSSI
(Empirical)', linewidth=2, markersize=8)
    plt.plot(d_model, theoretical_rssi, 'b--', label='Theoretical Friis FSPL Model
(2.4 GHz)', linewidth=1.5)

    plt.title(f"Live Wi-Fi Signal Decay Curve for SSID: '{target_ssid}'", fontsize=12,
fontweight='bold')
    plt.xlabel("Distance from Router (meters)", fontsize=10)
    plt.ylabel("Received Signal Strength Indicator RSSI (dBm)", fontsize=10)
    plt.axhline(-67, color='orange', linestyle=':', label='Quality VoIP Threshold (-67
dBm)')
    plt.grid(True, linestyle=':', alpha=0.6)
    plt.legend()
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()

```

6. Conclusion

- Live RSSI readings fluctuate due to indoor multi-path fading, requiring averaging over multiple samples at each spatial location.

- The real-time experimental data curve tracks the general inverse-square / logarithmic attenuation path of the theoretical Friis model, displaying increased path loss (dBm drop) as distance doubles.

Experiment 6: Analysis of channel interference in the 2.4 GHz band.

1. Aim

To capture real-time Wi-Fi channel data from surrounding Access Points (APs) in the 2.4 GHz ISM band using Python, analyze real-time Co-Channel Interference (CCI) and Adjacent-Channel Interference (ACI), and dynamically plot the live Power Spectral Density (PSD) graph.

2. Software Requirement

- **Programming Environment:** Python 3.x
- **Required Libraries:**
 - pywifi: To interface with the host system's wireless network adapter and scan real-time APs.
 - NumPy: To construct real-time Gaussian spectral masks.
 - Matplotlib: For real-time spectral visualization.
- **Operating System:** Windows or Linux (with an active, enabled Wi-Fi card).

Software Installation Command:

```
pip install pywifi numpy matplotlib
```

3. Theory

3.1 Live Signal Intercept & Spectral Reconstruction

Real-time Wi-Fi scanners retrieve metadata from broadcast Beacon Frames transmitted by surrounding Access Points. The key parameters extracted during a live scan are:

1. **SSID:** Broadcast network name.
2. **Channel Center Frequency (f_c):** Calculated directly from the channel number ($f_c = 2412 + 5 \times (\text{Channel} - 1) \text{ MHz}$).
3. **Received Signal Strength Indicator (RSSI in dBm):** Represents actual received signal power level.

3.2 Real-time Power Spectral Density (PSD) Modeling

Instead of using fixed unit amplitude curves, each live AP's spectral footprint is dynamically scaled according to its actual received power (RSSI in dBm). Converting RSSI from dBm to linear milliwatts (mW):

$$P_{mW} = 10^{(RSSI / 10)}$$

The real-time power spectrum $S(f)$ of each active channel centered at f_c with a 20 MHz channel bandwidth ($\sigma \approx 20 / 2.355$) is modeled as:

$$S(f) = P_{mW} \exp(-0.5 \cdot ((f - f_c)/\sigma)^2)$$

4. Algorithm / Experimental Procedure

Procedure Flowchart:

```
[Initialize Wi-Fi Adapter via PyWiFi]
-> Step 1: Scan surrounding 2.4 GHz channels for active Access Points
-> Step 2: Extract Live SSID, Channel Number, and RSSI (dBm)
-> Step 3: Map Channel Numbers to center frequencies (2412 MHz to 2462 MHz)
-> Step 4: Convert RSSI (dBm) -> Linear Power (mW)
-> Step 5: Compute real-time spectral curve  $S(f)$  for every detected AP
-> Step 6: Plot Live 2.4 GHz Interference Spectrum
```

Real-Time Python Implementation Code:

```

import time
import numpy as np
import matplotlib.pyplot as plt
import pywifi
from pywifi import const

# Step 1: Initialize PyWiFi Interface
def get_wifi_scanner():
    wifi = pywifi.PyWiFi()
    if len(wifi.interfaces()) == 0:
        raise RuntimeError("No Wi-Fi adapter detected on this system!")
    iface = wifi.interfaces()[0] # Select primary Wi-Fi card
    return iface

# Step 2: Perform Real-Time Channel Scan
def scan_realtime_networks(iface):
    print("Initiating real-time 2.4 GHz spectrum scan...")
    iface.scan()
    time.sleep(3) # Wait for scan buffer to populate
    scan_results = iface.scan_results()
    ap_list = []

    # 2.4 GHz Frequency to Channel Mapping
    freq_to_chan = {2412 + 5 * (ch - 1): ch for ch in range(1, 12)}

    for ap in scan_results:
        # Filter strictly for 2.4 GHz band (2412 MHz - 2462 MHz)
        freq_mhz = ap.freq / 1000 if ap.freq > 10000 else ap.freq # Handle kHz/MHz
        variance
        if freq_mhz in freq_to_chan:
            channel = freq_to_chan[freq_mhz]
            ssid = ap.ssid if ap.ssid else "Hidden_AP"
            rssi = ap.signal # RSSI in dBm
            ap_list.append({"ssid": ssid, "channel": channel, "freq": freq_mhz,
"rssi": rssi})
            print(f"Detected AP: SSID='{ssid}', Channel={channel}, RSSI={rssi} dBm")

    return ap_list

# Step 3: Spectral Curve Calculation
def generate_spectral_curve(freq_axis, center_freq, rssi_dbm, bw=20):
    # Convert RSSI (dBm) to Linear Power (mW) for realistic scale
    power_mw = 10 ** (rssi_dbm / 10.0)
    sigma = bw / 2.355
    return power_mw * np.exp(-0.5 * ((freq_axis - center_freq) / sigma) ** 2)

# Step 4: Main Execution and Real-Time Plotting
def main():
    try:
        iface = get_wifi_scanner()
        networks = scan_realtime_networks(iface)
        if not networks:
            print("No 2.4 GHz Access Points detected. Ensure Wi-Fi is enabled.")
            return

```

```

# Frequency spectrum array (2.400 GHz to 2.485 GHz)
freq_axis = np.linspace(2400, 2485, 1000)
plt.figure(figsize=(11, 6))

# Plot spectral mask for every detected real-time Access Point
for net in networks:
    spectral_mask = generate_spectral_curve(freq_axis, net["freq"],
net["rssi"])
    label_text = f"{net['ssid']} (Ch {net['channel']}, {net['rssi']} dBm)"
    plt.plot(freq_axis, spectral_mask, label=label_text, linewidth=2)
    plt.fill_between(freq_axis, 0, spectral_mask, alpha=0.2)

# Highlight standard non-overlapping channels (1, 6, 11) for reference
plt.axvline(2412, color='green', linestyle='--', alpha=0.5, label='Ch 1
Center')
plt.axvline(2437, color='green', linestyle='--', alpha=0.5, label='Ch 6
Center')
plt.axvline(2462, color='green', linestyle='--', alpha=0.5, label='Ch 11
Center')

plt.title("Real-Time 2.4 GHz Wi-Fi Channel Interference & Power Spectrum",
fontsize=12, fontweight='bold')
plt.xlabel("Frequency (MHz)", fontsize=10)
plt.ylabel("Received Power (milliwatts mW)", fontsize=10)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper right', bbox_to_anchor=(1.35, 1.0))
plt.tight_layout()
plt.show()

except Exception as e:
    print(f"Error executing real-time scan: {e}")

if __name__ == "__main__":
    main()

```

5. Conclusion

1. The real-time Python script successfully interfaces with physical Wi-Fi hardware using pywifi to intercept active beacon signals across the 2.4 GHz band.
2. Real-time RSSI readings (dBm) provide a live quantitative measure of signal power, displaying how nearby overlapping Access Points introduce live Adjacent-Channel Interference (ACI) and Co-Channel Interference (CCI) into the localized RF environment.

Experiment 7: Throughput and latency measurement using network tools (e.g., ping, speed test). (Done in Linux)

1. Aim

To measure, evaluate, and analyze network performance metrics—specifically Latency (Round-Trip Time - RTT), Jitter, and Data Throughput (Download/Upload Speeds)—using standard command-line tools (ping, traceroute/mtr) and throughput tools (iperf3, speedtest-cli) in a Linux environment.

2. Software / System Requirement

- **Operating System:** Linux (Ubuntu/Debian/Fedora/RHEL or any Linux terminal environment)
- **Pre-installed CLI Utilities:** ping, iproute2 (ip)
- **Package Requirements:** iperf3, speedtest-cli, mtr

Software Installation Command:

```
sudo apt update && sudo apt install -y iperf3 speedtest-cli mtr # Ubuntu/Debian
# OR
sudo dnf install -y iperf3 speedtest-cli mtr # Fedora/RHEL
```

3. Theory

3.1 Network Metrics Overview

1. **Latency (Round-Trip Time - RTT):** The time taken (in milliseconds, ms) for a data packet to travel from a source host to a destination host and return a response.
2. **Jitter:** The statistical variance in packet arrival latency over time. Low jitter is essential for real-time applications like VoIP and online video streaming.
3. **Throughput:** The actual rate at which data is successfully transmitted over a communication channel per unit of time, typically expressed in Mbps (Megabits per second) or Gbps.
4. **Packet Loss:** The percentage of transmitted data packets that fail to reach their destination, indicating network congestion, hardware faults, or poor RF conditions.

3.2 Mathematical Formulas

- **Average Latency (RTT_{avg}):**

$$RTT_{avg} = (1/N) \sum RTT_i$$

- **Packet Loss Percentage (%L):**

$$\%L = ((P_{tx} - P_{rx}) / P_{tx}) \times 100$$

- **Throughput (T):**

$$T = \text{Total Bytes Transferred (bits)} / \text{Total Transfer Time (seconds)}$$

4. Algorithm / Experimental Procedure

Procedure Flowchart:

```
[Start Linux Terminal]
-> Part A: Measure RTT & Packet Loss via ICMP (ping)
-> Part B: Trace Multi-Hop Path & Latency Degradation (mtr)
-> Part C: Measure Local LAN Throughput via TCP/UDP (iperf3)
-> Part D: Measure WAN Internet Throughput (speedtest-cli)
[Log Data & Plot Analysis]
```

Part A: Latency and Packet Loss Measurement using ping

1. Basic Ping Test (5 Packets):

```
ping -c 5 google.com
```

2. Adjusted Interval and Packet Payload Size: Test large payloads (1000 bytes) sent every 0.2 seconds:

```
ping -c 20 -i 0.2s -s 1000 8.8.8.8
```

Expected Terminal Output (Log Sample):

```
PING 8.8.8.8 (8.8.8.8) 1000 (1028) bytes of data.
1028 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=14.2 ms
1028 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=13.8 ms
1028 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=15.1 ms

--- 8.8.8.8 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 3804ms
rtt min/avg/max/mdev = 13.812/14.504/16.210/0.712 ms
```

Part B: Point-to-Point Path and Hop Analysis using mtr

Execute a dynamic traceroute/ping combination tool to analyze per-hop packet drop and latency accumulation:

```
mtr -c 10 --report 8.8.8.8
```

Part C: Local LAN Network Throughput Measurement using iperf3

iperf3 operates in a client-server architecture to measure maximum achievable TCP and UDP throughput without internet provider limitations.

1. On Server Node (Receiver Host - IP: 192.168.1.50):

```
iperf3 -s
```

2. On Client Node (Sender Host):

- TCP Throughput Test (10-second duration):

```
iperf3 -c 192.168.1.50 -t 10
```

- Parallel Stream Test (4 Parallel Connections):

```
iperf3 -c 192.168.1.50 -P 4
```

- UDP Jitter & Packet Loss Test (Targeting 100 Mbps bandwidth):

```
iperf3 -c 192.168.1.50 -u -b 100M
```

Expected Client Output (iperf3 TCP):

[ID]	Interval	Transfer	Bitrate	Retr	
[5]	0.00-10.00 sec	1.10 GBytes	944 Mbits/sec	0	sender
[5]	0.00-10.04 sec	1.10 GBytes	940 Mbits/sec		receiver

Part D: Public WAN Internet Speed Measurement using speedtest-cli

Execute a standardized bandwidth test against a public server:

```
speedtest-cli --simple
```

Output Format:

```
Ping: 12.45 ms
Download: 94.20 Mbit/s
Upload: 88.15 Mbit/s
```

5. Conclusion

Network performance metrics were measured and analyzed using Linux CLI tools:

1. `ping` verified network reachability, showing an average latency (RTT_{avg}) of 14.5 ms with 0% packet loss.
2. `iperf3` demonstrated maximum raw TCP local network throughput of ≈ 940 Mbps on a Gigabit Ethernet link, verifying layer-4 data transmission rates.

3. `speedtest-cli` quantified WAN connectivity speeds, highlighting the performance difference between internal local bandwidth and external internet service provider (ISP) limits.

Linux Commands for experiment 7

Network latency and throughput are measured using distinct protocols. Latency (round-trip time) is best measured with ping, while throughput (bandwidth/speed) requires tools like iperf3 or speedtest-cli.

1. Measuring Latency (Using ping)

The ping command sends ICMP Echo Request packets to a target IP or domain to measure Round-Trip Time (RTT) in milliseconds (ms).

- **Basic ping test:** `ping -c 5 google.com` (Limits the output to 5 packets).
Verification: Look at the summary line at the bottom (rtt min/avg/max/mdev = ...). The avg value represents your average latency.
- **Adjusting interval or payload size:** `ping -c 10 -i 0.5s -s 1000 8.8.8.8`
 - i 0.5 : Sends a packet every 0.5 seconds.
 - s 1000 : Changes packet size to 1000 bytes.

2. Measuring Throughput in a Local Network (Using iperf3)

iperf3 directly tests raw TCP/UDP throughput between two Linux hosts without internet overhead.

- **Install iperf3:** `sudo apt install iperf3` (Debian/Ubuntu) or `sudo dnf install iperf3` (RHEL/Fedora). Verification: Run `iperf3 -v`
- **Start the Server Machine:** On Machine A (the receiver): `iperf3 -s`. Verification: Ensure it outputs "Server listening on 5201".
- **Run the Test from the Client Machine:** On Machine B (sender): `iperf3 -c <SERVER_IP> -t 10`. Verification: Read the final summary row under the Bitrate column.
- **Test Reverse Throughput (Download):** `iperf3 -c <SERVER_IP> -R`
- **Test Parallel Streams:** `iperf3 -c <SERVER_IP> -P 4`

3. Measuring Internet Speed (Using speedtest-cli)

- **Install speedtest-cli:** `sudo apt install speedtest-cli`
- **Run a quick speed test:** `speedtest-cli --simple`

Experiment 8: Study of Bluetooth communication using mobile devices (pairing, range testing).

1. Aim

To study the principles of Bluetooth short-range wireless communication by performing device pairing, link establishment, and empirical range testing to evaluate signal degradation, packet loss, and maximum operational throughput over distance.

2. Software / Hardware Requirement

• Hardware:

- Two Bluetooth-enabled smartphones (Android/iOS) or one smartphone and a Bluetooth low energy (BLE) peripheral/development board (e.g., ESP32/HC-05).
- Measuring tape or laser rangefinder (0 to 50 meters).

• Software Apps:

- Android/iOS: Serial Bluetooth Terminal, nRF Connect for Mobile (Nordic Semiconductor), or Bluetooth Serial Controller.
- Data Logging: Built-in Bluetooth Settings / System Diagnostics.

3. Theory

3.1 Bluetooth Architecture & Frequency Allocation

Bluetooth operates in the 2.4 GHz ISM (Industrial, Scientific, and Medical) band ranging from 2.402 GHz to 2.480 GHz. To prevent interference from co-located Wi-Fi networks and microwave ovens, Bluetooth uses Frequency Hopping Spread Spectrum (FHSS), hopping across sub-channels 1,600 times per second.

- **Classic Bluetooth (BR/EDR):** Uses 79 channels spaced 1 MHz apart. Designed for continuous, high-throughput streams (e.g., audio, large file transfers).
- **Bluetooth Low Energy (BLE):** Uses 40 channels spaced 2 MHz apart (3 Advertising channels and 37 Data channels). Optimized for low power consumption and bursty data transmissions.

3.2 Power Classes and Range Specifications

Bluetooth radios are categorized into power classes that dictate nominal range:

Class	Maximum Output Power	Typical Range (Line-of-Sight)
Class 1	100 mW (+20 dBm)	Up to 100 meters
Class 2	2.5 mW (+4 dBm)	Up to 10 meters (Standard mobile devices)
Class 3	1 mW (0 dBm)	Up to 1 meter

3.3 Pairing and Connection Establishment

- 1. **Inquiry/Discovery:** The master device broadcasts inquiry packets across advertising channels.
- 2. **Paging:** The master connects to a specific discovered slave device using its BD_ADDR (Bluetooth Device Address).
- 3. **Authentication & Bonding:** Devices exchange security keys (Passkey / Just Works / Numeric Comparison) to encrypt the link layer.

4. Algorithm / Experimental Procedure

Procedure Flowchart:

```
[Start]
-> Step 1: Enable Bluetooth on both mobile devices (Device A Master, Device B Slave)
-> Step 2: Make Device B discoverable and initiate Discovery scan from Device A
-> Step 3: Select Device B, execute Pairing/Bonding, and establish RFCOMM/GATT connection
-> Step 4: Open 'Serial Bluetooth Terminal' or 'nRF Connect' app on both devices
-> Step 5: Perform Line-of-Sight (LoS) Range Test at 1m, 5m, 10m, 15m, 20m, 30m (Measure RSSI,
-> Step 6: Perform Non-Line-of-Sight (NLOS) Range Test through physical obstacles (walls)
-> Step 7: Record connection drop threshold (Out-of-Range point)
[End]
```

Experimental Data Collection Table:

Distance (d in meters)	Environment Type	Average RSSI (dBm)	Data Transfer Status	Latency / Ping (ms)	Link State
1 m	Line-of-Sight (LoS)	-42 dBm	100% Delivered	15 ms	Connected (Stable)
5 m	Line-of-Sight (LoS)	-58 dBm	100% Delivered	18 ms	Connected (Stable)
10 m	Line-of-Sight (LoS)	-71 dBm	98% Delivered	25 ms	Connected (Stable)
15 m	Line-of-Sight (LoS)	-82 dBm	85% Delivered	65 ms	High Jitter / Packet Retries
20 m	Line-of-Sight (LoS)	-91 dBm	40% Delivered	>200 ms	On the verge of disconnect
23 m	Line-of-Sight (LoS)	-96 dBm	0% Delivered	∞	Connection Dropped
8 m	Non-LoS (Concrete Wall)	-86 dBm	70% Delivered	110 ms	Weak Link

5. Conclusion

1. Bluetooth link establishment follows a structured Inquiry → Page → Authentication → Connection sequence.
2. Class 2 mobile Bluetooth devices maintain a reliable link up to ≈ 10 -12 meters in line-of-sight conditions. Beyond 15 meters, Received Signal Strength Indicator (RSSI) degrades below -85 dBm, causing significant frame retransmissions and latency spikes.
3. Physical obstacles such as concrete walls introduce severe attenuation (≈ 12 -18 dBm loss), drastically reducing the effective coverage radius compared to free-space propagation.